

Unit Testing Guide — WebMessenger

Подготовка к экзамену по навыкам: xUnit · Moq · Bogus · AutoFixture · Fixtures · Theory · Debugging · Manual API

Validation Проект: WebMessenger.Api.Tests / WebMessenger.Contracts.Tests · .NET 8 · xUnit 2.9 · Moq 4.20 · Bogus 35.6

1. Зачем нужны тесты?

Представь: ты меняешь `AuthService.ValidateUserCredentials`. Без тестов ты узнаешь, что сломал логин, только когда вручную зайдёшь в браузер. С тестами — через 3 секунды после сохранения файла.

- **Уверенность при рефакторинге** — сломал что-то → тест упал сразу
- **Документация** — читая тест, понимаешь что должен делать метод
- **Изоляция ошибок** — тест говорит точно что упало и почему
- **Скорость** — 76 тестов за 4 секунды vs 30 минут ручной проверки

2. Паттерн AAA — основа всего

AAA = Arrange → Act → Assert. Каждый тест делится на три чётких блока.



```
// AuthServiceTests.cs
[Fact] public void
ValidateUserCredentials_ValidPassword_ReturnsTrue()
{
    // Arrange — подготовка: создаём объекты, настраиваем
    данные    const string rawPassword = "P@ssw0rd!";    var user
    = UserFaker.Single();
    user.PasswordHash = BCrypt.HashPassword(rawPassword);

    // Act — вызов тестируемого метода (ровно одна строка)
    var result = _sut.ValidateUserCredentials(user, rawPassword);

    // Assert — проверка результата
    Assert.True(result);
}
```

`_sut` = *System Under Test* — конвенция: переменная с тестируемым объектом. Сразу ясно что именно тестируется.

Именование тестов

Паттерн: `МетодКоторыйТестируем_Условие_ОжидаемыйРезультат`

Пример

Что означает	
<code>ValidateUserCredentials_ValidPassword_ReturnsTrue</code>	правильный пароль → true
<code>ValidateUserCredentials_NullUser_ReturnsFalse</code>	null пользователь → false

<code>GenerateJwtToken_NullUser_ThrowsArgumentNullException</code>	null → исключение
<code>Register_ExistingUsername_ReturnsBadRequest</code>	дубликат → 400

3. [Fact] vs [Theory]

[Fact] — тест без параметров, запускается один раз

```
[Fact]
public void GenerateJwtToken_ValidUser_ReturnsNonEmptyToken()
{
    var user = UserFaker.Single();
    var token = _sut.GenerateJwtToken(user);

    Assert.False(string.IsNullOrEmpty(token));
    Assert.Equal(3, token.Split('.').Length); // JWT = header.payload.signature
}
```

[Theory] + [InlineData] — один тест, несколько наборов данных

```
[Theory]
[InlineData("")]
[InlineData(" ")]
[InlineData("\t")] public void
ValidateUserCredentials_EmptyOrWhitespacePassword_ReturnsFalse(string password)
{
    var user =
UserFaker.Single();
    var result = _sut.ValidateUserCredentials(user, password);
    Assert.False(result);
}
```

xUnit запустит этот тест трижды, подставляя каждое значение. В Test Explorer видно как три отдельные строки:

```
ValidateUserCredentials_EmptyOrWhitespacePassword_ReturnsFalse(password: "")
ValidateUserCredentials_EmptyOrWhitespacePassword_ReturnsFalse(password: " ")
ValidateUserCredentials_EmptyOrWhitespacePassword_ReturnsFalse(password: "\t")
```

Используй **[Theory]** для: граничных значений (пустая строка, null, макс. длина), нескольких ID пользователей, разных форматов входных данных.

4. Moq — изоляция зависимостей

Проблема без моков

`UserService` зависит от `IUnitOfWork`, `IContactsService`, `IAuthService`. Если тестировать с реальными объектами — нужна настоящая база данных, реальный JWT ключ. Тест перестаёт быть *unit* тестом.

Мок — это поддельный объект, реализующий интерфейс

```
// Создание var _userRepoMock = new
Mock<IRepository<User>>();

// Setup — настраиваем поведение
_userRepoMock.Setup(r => r.GetAll(It.IsAny<string[]>()))
    .Returns(new[] { user }.AsAsyncQueryable());

// Setup для async методов
_userRepoMock.Setup(r => r.GetAsync(user.Id, It.IsAny<CancellationToken>()))
    .ReturnsAsync(user);

// Setup исключения
_userRepoMock.Setup(r => r.GetAsync(It.IsAny<Guid>(), It.IsAny<CancellationToken>()))
    .ThrowsAsync(new Exception("DB down"));
```

Verify — проверка вызовов

```
// Из UserServiceTests — после регистрации должны быть вызваны InsertAsync и CommitAsync
_userRepoMock.Verify(
    r => r.InsertAsync(It.Is<User>(u => u.Username == dto.Username), It.IsAny<CancellationToken>()),
    Times.Once
);
_uowMock.Verify(u => u.CommitAsync(It.IsAny<CancellationToken>()), Times.Once);

// Проверяем что RegisterUserAsync НЕ был вызван при дубликате
_userMock.Verify(s => s.RegisterUserAsync(It.IsAny<RegisterDto>()), Times.Never);
```

Матчер

Значение

<code>It.IsAny<T>()</code>	любое значение типа T
<code>It.Is<T>(x => x.Prop == val)</code>	значение с условием
<code>Times.Once</code>	ровно один раз
<code>Times.Never</code>	никогда не вызывался
<code>Times.AtLeastOnce</code>	хотя бы один раз
<code>Times.Exactly(n)</code>	ровно n раз

UnitOfWorkMockHelper — переиспользуемые моки

```
// Shared/Mocks/UnitOfWorkMockHelper.cs
public static Mock<IUnitOfWork> Create()
{
    var mock = new Mock<IUnitOfWork>(); mock.Setup(u => u.UserRepository).Returns(new
Mock<IRepository<User>>().Object); mock.Setup(u =>
u.CommitAsync(It.IsAny<CancellationToken>())).Returns(Task.CompletedTask);
    // ... все репозитории
    return mock;
}

// В тесте: создаём базу и переопределяем нужный репозиторий
_uowMock = UnitOfWorkMockHelper.Create();
_uowMock.Setup(u => u.UserRepository).Returns(_userRepoMock.Object);
```

5. Fixtures — переиспользование дорогих объектов

IClassFixture — один объект на весь тестовый класс

```
// UserPoolFixture.cs — создаётся ОДИН РАЗ для всего класса
public sealed class UserPoolFixture
{
    public IReadOnlyList<User> Users { get; } = UserFaker.Many(10, seed:
42); }

// AuthServiceTheoryTests.cs — принимает fixture через конструктор
public class AuthServiceTheoryTests : IClassFixture<UserPoolFixture>
{
    private readonly IReadOnlyList<User>
_users;

    public AuthServiceTheoryTests(UserPoolFixture fixture)
    {
        _users = fixture.Users; // используем готовых пользователей
    }

    [Theory]
    [InlineData(0)]
    [InlineData(3)] [InlineData(7)] public void
GetUsernameFromToken_TokenGeneratedForPooledUser_ReturnsCorrectUsername(int userIndex)
    {
        var user = _users[userIndex];
var token = _sut.GenerateJwtToken(user);
        var username = _sut.GetUsernameFromToken($"Bearer {token}");
        Assert.Equal(user.Username, username);
    }
}
```

ICollectionFixture — один объект для нескольких классов

```
[CollectionDefinition(UserPoolCollection.Name)] public sealed class
UserPoolCollection : ICollectionFixture<UserPoolFixture>
{
    public const string Name =
"UserPool";
}

// Применяем к тестовому классу:
[Collection(UserPoolCollection.Name)]
public class SomeOtherTests { ... }
```

Вид	Жизненный цикл	Применение
Конструктор класса	Новый объект перед каждым тестом	Дешёвые объекты
IClassFixture<T>	Один объект на класс	Дорогие объекты (BCrypt, JWT)
ICollectionFixture<T>	Один объект на несколько классов	Общий пул данных

Правило: Fixture — только для чтения. Никогда не мутируй данные внутри fixture из теста, иначе тесты станут зависимыми друг от друга (fixture leakage).

6. Генерация данных — Bogus и AutoFixture

Без Bogus — скучно и хрупко

```
var user = new User { Id = Guid.NewGuid(), Username = "testuser", PasswordHash = "hash", /* ещё 10 полей */ };
```

Bogus — реалистичные фейковые данные с фиксированным seed

```
// UserFaker.cs public static Faker<User> Create(int
seed = 1337) => new Faker<User>()
    .UseSeed(seed) // фиксируем seed → детерминизм
    .RuleFor(u => u.Id, f => f.Random.Guid())
    .RuleFor(u => u.Username, f => f.Internet.UserName()) // "john.doe42"
    .RuleFor(u => u.Email, f => f.Internet.Email()) // "john@example.com"
    .RuleFor(u => u.FirstName, f => f.Name.FirstName()) // "Alice"
    .RuleFor(u => u.LastName, f => f.Name.LastName()); // "Smith"

UserFaker.Single(seed: 1337) // один пользователь
UserFaker.Many(10, seed: 42) // список из 10
```

Детерминизм — ключевой момент

```
var user1 = UserFaker.Single(seed: 999);
var user2 = UserFaker.Single(seed: 999);
Assert.Equal(user1.Username, user2.Username); // всегда одинаковые
```

Ловушка: `Randomizer.Seed = new Random(1337)` — глобальное состояние, ломает параллельные тесты.

Правильно: `new Faker<T>().UseSeed(1337)` — per-instance seed, изолированный.

AutoFixture + AutoMoq

```
// FixtureFactory.cs var fixture = new Fixture(); fixture.Customize(new
AutoMoqCustomization { ConfigureMembers = true });

// Автоматически создаёт объекты и подставляет моки для всех интерфейсов
var service = fixture.Create<UserService>();
```

7. Тестирование контроллеров — статус коды

Контроллеры тестируем без HTTP запросов, напрямую вызывая методы:

```
// AuthControllerTests.cs
public AuthControllerTests()
{
    _authMock = new Mock<IAuthService>();
    _userMock = new Mock<IUserService>();
    _sut = new AuthController(_authMock.Object, _userMock.Object, NullLogger<AuthController>.Instance);
    // Нужен DefaultHttpContext для Response.Cookies.Append
    _sut.ControllerContext = new ControllerContext { HttpContext = new DefaultHttpContext() };
}

[Fact] public async Task
Login_ValidCredentials_ReturnsOkWithToken()
{
    // Arrange
    var user = UserFaker.Single();
    _userMock.Setup(s => s.FindUserByUsernameAsync(user.Username)).ReturnsAsync(user);
    _authMock.Setup(s => s.ValidateUserCredentials(user, "correct")).Returns(true);
    _authMock.Setup(s => s.GenerateJwtToken(user)).Returns("jwt.token.value");

    // Act    var result = await _sut.Login(new LoginDto { Username = user.Username, Password =
    "correct" });

    // Assert
    var ok = Assert.IsType<OkObjectResult>(result);
    Assert.NotNull(ok.Value);
    _authMock.Verify(s => s.GenerateJwtToken(user), Times.Once);
}
```

Assert

HTTP статус

<code>Assert.IsType<OkObjectResult>(result)</code>	200 OK
<code>Assert.IsType<BadRequestObjectResult>(result)</code>	400 Bad Request
<code>Assert.IsType<UnauthorizedObjectResult>(result)</code>	401 Unauthorized
<code>Assert.IsType<NotFoundResult>(result)</code>	404 Not Found
<code>Assert.IsType<ObjectResult>(r); Assert.Equal(500, r.StatusCode)</code>	500 Server Error

8. Валидация DTO и Error Handling

```
// ControllerErrorHandlingTests.cs – все ошибочные пути в одном файле
[Fact] public async Task
UserController_UploadAvatar_NoFile_Returns400()
{
    var result = await sut.UploadAvatar(null!);
    Assert.IsType<BadRequestObjectResult>(result);
}

[Fact] public async Task
UserController_UploadAvatar_ServiceFails_Returns500()
{
    avatarMock.Setup(s =>
s.UpdateUserAvatarAsync(...)).ReturnsAsync((string?)null);    var result = await
sut.UploadAvatar(file.Object);    var obj = Assert.IsType<ObjectResult>(result);
    Assert.Equal(500, obj.StatusCode);
}
```

Сценарий	Ожидаемый статус
null файл для аватара	400
Неверный MIME тип (PDF вместо image/)	400
Файл больше 5MB	400
Сервис вернул null (ошибка загрузки)	500
Пустой поисковый запрос	400
Дубликат username при регистрации	400
Добавить себя в контакты	400
Неверный пароль при логине	401

9. Debugging и ITestOutputHelper

```
// DebuggingShowcaseTests.cs public class
DebuggingShowcaseTests(ITestOutputHelper output)
{
    private readonly TestLogger _log =
    new(output);

    [Fact] public void
    MockInteraction_LogsCallsForDebugging()
    {
        var contactsMock = new Mock<IContactsService>();
        contactsMock.Setup(c => c.IsContact(userId, contactId)).Returns(true);

        _log.Log("Calling IsContact with userId={0}", userId); // видно в Test Explorer

        var result = contactsMock.Object.IsContact(userId, contactId);

        _log.Log("Result: {0}", result); Assert.True(result);
        contactsMock.Verify(c => c.IsContact(userId, contactId), Times.Once);
    }
}
```

`Console.WriteLine` не работает в Test Explorer. Используй `ITestOutputHelper` — вывод виден только при упавшем тесте.

Debugging Playbook — 4 типичные ошибки

Проблема	Симптом	Решение
Неверный Setup	мок возвращает null/default вместо значения	Проверь что matcher совпадает с реальным вызовом (аргументы, типы)
Fixture leakage	тест проходит один, падает в suite	Не мутируй fixture. <code>IClassFixture</code> — только чтение
Async timing	intermittent failures	Всегда <code>await</code> , никогда <code>.Result</code> или <code>.Wait()</code>
Non-deterministic data	флаку тесты на разных машинах	Используй <code>UseSeed()</code> в Bogus

Демонстрация неверного Setup



```
[Fact] public async Task
WrongMockSetupDiagnosis_IncorrectArgMatcher_MockReturnsDefault()
{
    var specificId =
    Guid.NewGuid();    // Setup только
    для specificId
    userRepoMock.Setup(r => r.GetAsync(specificId, ...)).ReturnsAsync(user);

    // Вызов с ДРУГИМ id – мок не матчит, возвращает
    null    var differentId = Guid.NewGuid();    var
    result = await repo.GetAsync(differentId);

    Assert.Null(result); // null потому что setup не совпал – типичная ловушка!
    // Исправление: использовать It.IsAny<Guid>() вместо specificId
}
```

10. Manual API Validation (Swagger / Postman)

Чеклист для каждого эндпоинта

Что проверять	Как
Success case	правильные данные → ожидаемый статус и тело ответа
Failure case	неверные credentials → 401
Negative payload	пустое поле, неверный тип → 400
Auth required	без токена → 401
Domain conflict	дубликат данных → 400 / 409

Примеры из runbook

```
// 1. Успешная регистрация → 200
POST /api/auth/register
{ "username": "testuser", "password": "P@ssw0rd1" }
→ 200: { "message": "Registration successful" }

// 2. Дубликат username → 400
POST /api/auth/register
{ "username": "testuser", "password": "anything" }
→ 400: { "message": "Username already exists" }

// 3. Успешный логин → 200 + токен
POST /api/auth/login
{ "username": "testuser", "password": "P@ssw0rd1" } →
200: { "token": "<jwt>" } + Set-Cookie: auth-token=...

// 4. Неверный пароль → 401
POST /api/auth/login
{ "username": "testuser", "password": "WRONG" }
→ 401: { "message": "Invalid credentials" }

// 5. Запрос без токена → 401 GET
/api/chats (без Authorization header)
→ 401 Unauthorized
```

11. Шпаргалка — Assert API (xUnit)

```
Assert.True(x) / Assert.False(x)
Assert.Equal(expected, actual) / Assert.NotEqual(a, b)
Assert.Null(x) / Assert.NotNull(x)
Assert.Empty(list) / Assert.Single(list)
Assert.IsType<OkObjectResult>(result)
Assert.Contains(list, x => x.Id == id)
Assert.DoesNotContain(list, x => x.Id == id)
Assert.Throws<ArgumentNullException>(() => sut.Method(null))
await Assert.ThrowsAsync<Exception>(() => sut.AsyncMethod())
```

12. Структура проекта

```
WebMessenger.Api.Tests/
├─ Shared/
│   ├── AsyncQueryableExtensions.cs ← EF Core async поддержка в тестах
│   ├── TestLogger.cs ← обёртка над ITestOutputHelper
│   ├── UserPoolFixture.cs ← IClassFixture + ICollectionFixture
│   └─ Mocks/
│       └─ UnitOfWorkMockHelper.cs ← переиспользуемые моки UoW
│   └─ TestData/
│       ├── UserFaker.cs ← Bogus с UseSeed() – детерминизм
│       └─ DeterministicBogus.cs ← generic хелпер
├─ Unit/
│   ├── Smoke/SmokeTests.cs ← проверка что xUnit работает
│   ├── Services/
│   │   ├── AuthServiceTests.cs ← [Fact], AAA, BCrypt
│   │   ├── AuthServiceTheoryTests.cs ← [Theory], IClassFixture
│   │   ├── UserServiceTests.cs ← Moq Setup/Verify, async EF
│   │   └─ ContactsServiceTests.cs ← Moq, sync + async
│   ├── Controllers/
│   │   ├── AuthControllerTests.cs ← статус коды, Verify
│   │   ├── ContactControllerTests.cs
│   │   └─ UserControllerTests.cs ← IFormFile mock
│   ├── ErrorHandling/
│   │   └─ ControllerErrorHandlingTests.cs ← 400/401/500
│   └─ Debugging/
│       └─ DebuggingShowcaseTests.cs ← ITestOutputHelper, playbook
WebMessenger.Contracts.Tests/
├─ Unit/Validation/
│   ├── LoginDtoValidationTests.cs ← DTO проверки
│   └─ PagedResultTests.cs ← [Theory] граничные значения
```

WebMessenger Testing Guide · .NET 8 · xUnit 2.9 · Moq 4.20 · Bogus 35.6 · AutoFixture 4.18 Для
конвертации в PDF: открой в браузере → Ctrl+P → «Сохранить как PDF»